

ECE391

Computer System Engineering

Lecture 4

Prof. Kirill Levchenko and Dong Kai Wang

Slides adapted from Prof. Kalbarczyk

University of Illinois at Urbana- Champaign

Spring 2024

Administrivia

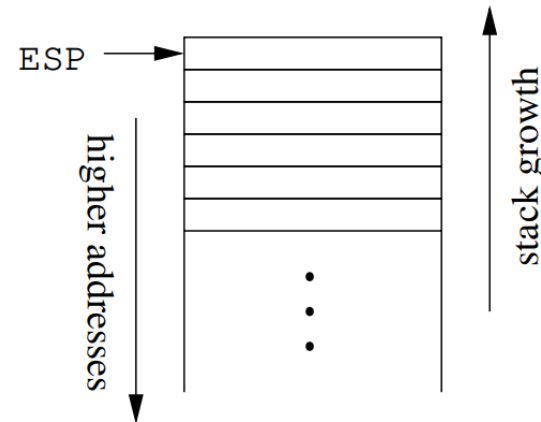
- When demoing, make sure your CA/TA has the form up and the right **NetID** to avoid any grading issues.
- MP1 was released yesterday, start early!

Lecture Topics

- Memory alignment
- Calling convention and stack frames
 - Example call assembly
- Role of systems software
- **Read course notes:**
 - Misc. x86 instructions (multiply, divide)
 - Syntax (label, comment rules, we didn't cover in class)

Recap Quiz

- Does the PUSH instruction increment or decrement the stack pointer (ESP)?



- Does POPL access $M[ESP]$ or $M[ESP+4]$?
- Why do we want to store 32-bit values at memory addresses that are multiples of 4?

Memory Alignment

- Caches / memory is organized in rows and columns.
 - Example: we want to store a mix of 2-byte and 4-byte data.

	12	8	4	0
		2B	Blank	4B Data
			2B	Blank
4B Data				

Example: 4-Byte Aligned, 16-Byte per Line

	14	12	10	8	6	4	2	0
				2B	4B Data			
							2B	4B
Data								

Example: 2-Byte Aligned, 16-Byte per Line **Needs 2 rows!**

Recall: Stack Operations

- Push and pop supported directly by x86 ISA

PUSHL %EAX # $M[ESP - 4] \leftarrow EAX, ESP \leftarrow ESP - 4$

POPL %EBP # $EBP \leftarrow M[ESP], ESP \leftarrow ESP + 4$

PUSHFL # $M[ESP - 4] \leftarrow EFLAGS, ESP \leftarrow ESP - 4$

POPFL # $EFLAGS \leftarrow M[ESP], ESP \leftarrow ESP + 4$

The Calling Convention (1)

- What is a calling convention?
 - generally: rules for subroutine interface structure.
 - specifically:
 - how information is passed into subroutine,
 - how information is returned to caller,
 - who owns registers.
 - often specified by vendor so that different compilers' code can work together (it's a **CONVENTION**).
- Parameters for subroutines:
 - pushed onto stack,
 - from **right to left** in C (*is lowest address left or rightmost parameter?*),
 - order can be dependent on language, compiler, OS.

The Calling Convention (2)

- Subroutine return values:
 - EAX for up to 32 bits.
 - EDX:EAX for up to 64 bits (quadword).
 - floating-point not discussed in this class (st0, ...).
- Register ownership:
 - return values can be clobbered by subroutine: EAX and EDX.
 - **caller-saved**: subroutine free to clobber; caller must preserve
 - ECX
 - EFLAGS
 - **callee-saved**: subroutine must preserve value passed in
 - stack structure: ESP and EBP
 - other registers: EBX, ESI, and EDI

Stack Frames in x86 (1)

- The call sequence

0. save caller-saved registers (if desired)

1. **push** arguments onto stack

2. make the call

3. **pop** arguments off the stack

4. restore caller-saved registers

Stack Frames in x86 (2)

- The **callee sequence** (creates the stack frame)
 0. save old base pointer and get new one
 1. save callee-saved registers (always why?)
 2. make space for local variables
 3. do the function body
 4. tear down stack frame (locals)
 5. restore callee-saved registers
 6. load old base pointer
 7. return *What does return correspond to?*

Stack Frames in x86 (3)

- Example of caller code (no caller-saved registers considered)

```
int func (int A, int B, int C);
```

```
func (100, 200, 300);
```



```
PUSHL $300  
PUSHL $200  
PUSHL $100  
CALL func  
ADDL $12,%ESP  
# result in EAX
```

Stack Frames in x86 (4)

- Example of subroutine code and stack frame creation and teardown

```
int func (int A, int B, int C)
{
    /* 12 bytes of local variables */
    ...
}
```

```
call func (100, 200, 300);
```

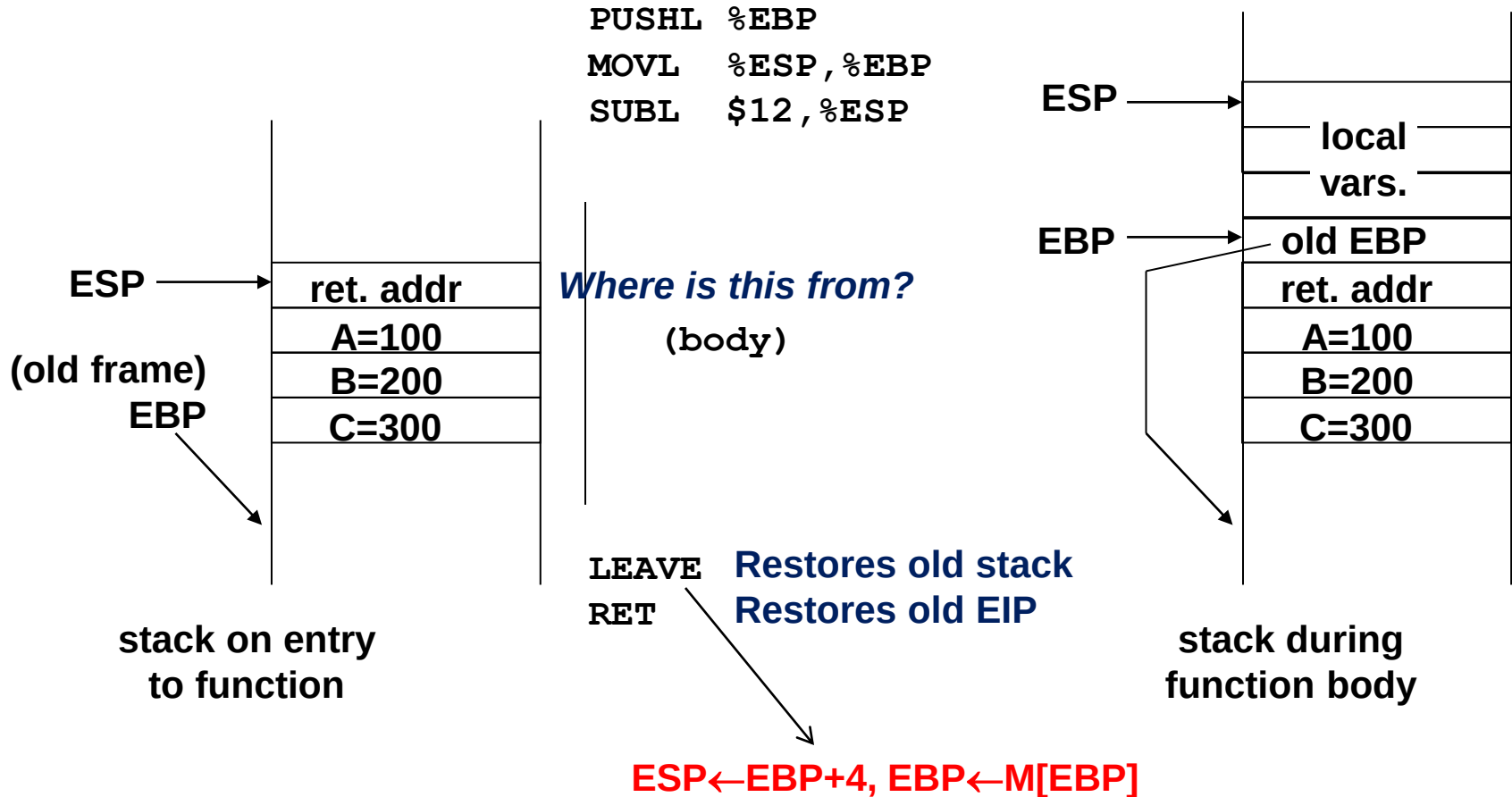
CALL printf

(push EIP), $EIP \leftarrow \text{printf}$

RET

$EIP \leftarrow M[ESP]$, $ESP \leftarrow ESP + 4$

Example



Subroutine Example Code

- Earlier assumptions (*from last lecture's min/max age code*):
 - some values start in registers (array pointer in EBX, length in ECX)
 - could specify output regs (min. age in EDX, max. age in EDI)

Turn it into a C function, we could write...

```
void find_min_max (person* group, long n_people,  
                  min_max* mm)
```

array of

char* name
long age

long min
long max

Q: how many bytes is the **group** argument? **mm** argument?

Q: how many bytes does **person** struct occupy? What about **min_max**?

Q: why is return **void**?

Subroutine Example Code (cont.)

{ step 1: create the stack frame

```
PUSHL %EBP
MOVL  %ESP,%EBP
PUSHL %EBX  # protect callee-saved
             # registers
PUSHL %ESI
PUSHL %EDI
```

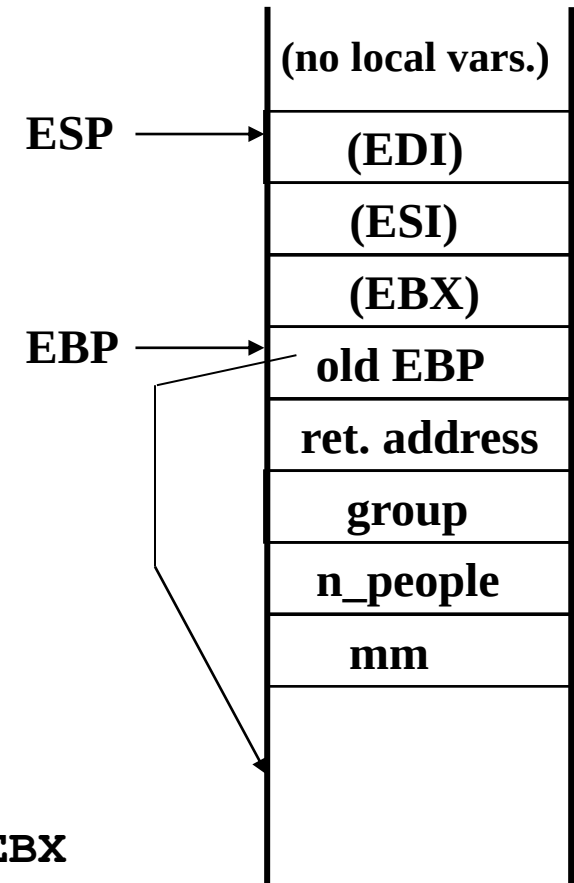
step 2: link to our input interface

```
MOVL  8(%EBP),%EBX  # group
MOVL  12(%EBP),%ECX # n_people
```

step 3: insert our code from before

step 4: link from our output interface

```
MOVL  16(%EBP),%EBX  # load mm into EBX
MOVL  %EDX,0(%EBX)   # mm <- min
MOVL  %EDI,4(%EBX)   # (mm + 4) <- max
```



Subroutine Example Code (cont.)

step 5: tear down stack frame

```
# we have no local variables to remove
# restore callee-saved registers
# (note that order is reversed!)
```

```
POPL %EDI
```

```
POPL %ESI
```

```
POPL %EBX
```

```
LEAVE
```

```
RET
```

alternate version (used by gcc)

```
LEAL -12(%EBP), %ESP
```

```
POPL %EDI
```

```
POPL %ESI
```

```
POPL %EBX
```

```
POPL %EBP
```

```
RET
```

